# AURA AI EMAIL INTELLIGENCE

## Production Readiness Test Audit & Phased Architecture Roadmap

**Document Type:** Production Engineering Assessment | **Target Release:** Commercial Public SaaS | **Current Version:** 1.0 Prototype

## 1. Executive Summary & Test Audit Results

An automated test audit of **web_app.py** and its underlying services was executed against active endpoints. The core LLM triage pipeline, atomic SQLite placeholder claims, and OAuth URL generation function successfully in a local single-user environment. However, **the application is currently architected as a single-user local prototype**. To publish this as a live, secure, commercial multi-tenant website, critical architectural changes are required to address session token leakage, lack of tenant database isolation, hardcoded user personality prompts, and Google Restricted Scope compliance.

| | | |
|---|---|---|
| **Current Prototype Readiness Score: 3.5 / 10** | **Target Go-Live Score: 9.8 / 10** | **Primary Blocker:** Google Restricted Scope Audit (CASA Tier 2) |

### Automated Endpoint Test Matrix

| Endpoint | Method | Test Result | Audit Finding & Defect Description | Risk |
|---|---|---|---|---|
| **GET /** | GET | **PASS (200 OK)** | Landing page renders properly with CSS and typography. Missing CSRF meta tag and Content Security Policy (CSP). | **LOW** |
| **GET /connect** | GET | **PASS (302 Redirect)** | Generates Google OAuth URL with PKCE. Relies on local *credentials.json*; fails if file missing in cloud container. | **MED** |
| **GET /callback** | GET | **CRITICAL RISK** | Stores entire OAuth credentials dict (client_secret, refresh_token) in client-side signed cookie. High risk of token compromise. | **CRIT** |
| **GET /dashboard** | GET | **PASS (302/200)** | Unauthenticated redirect works. Reactivates in-memory thread; will conflict in multi-worker production WSGI servers. | **HIGH** |
| **GET /run** | GET | **PASS (401/200)** | Returns 401 when unauthorized. When authorized, runs synchronous blocking LLM calls that risk Nginx gateway 504 timeouts. | **HIGH** |
| **GET /stats** | GET | **DATA LEAK** | Queries *processed_emails* without user_id filter. In multi-tenant deployment, User B sees User A's email statistics. | **CRIT** |
| **GET /replies** | GET | **DATA LEAK** | Returns all sent reply texts across all users from SQLite without account separation. Severe privacy breach risk. | **CRIT** |
| **GET /disconnect** | GET | **PASS (302)** | Clears local session cookie. Does not revoke OAuth token on Google OAuth servers. | **LOW** |

## 2. Critical Architectural Blockers & Security Vulnerabilities

Before publishing this web application to the public internet, the following six core flaws must be resolved to protect user data, ensure uptime, and avoid account suspension by Google.

### Blocker 1: Google Restricted Scopes Verification (CASA Assessment)

The app requests *https://www.googleapis.com/auth/gmail.send* and *gmail.readonly*. Because these access private user emails, Google immediately flags unverified apps with a severe warning ('Google hasn't verified this app') and caps access at 100 manual test accounts. Publishing to real users requires passing a third-party Cloud Application Security Assessment (CASA Tier 2 audit), hosting public legal policies, and completing official Google OAuth Verification.

### Blocker 2: Plaintext Hardcoded Secrets & Client-Side Token Storage

The Flask secret key is hardcoded to a default string ('change-this-to-random-string-123'), and *OAUTHLIB_INSECURE_TRANSPORT=1* permits plaintext HTTP transmission. Most critically, the full Google OAuth token dict (including refresh_token and client_secret) is serialized into the browser's session cookie. Storing long-lived refresh tokens in browser cookies violates OAuth security standards; tokens must be encrypted at rest in a server-side vault.

### Blocker 3: Hardcoded Single-User Personality & Logic

The system prompt is explicitly hardcoded to a single individual: *'You are an AI email assistant for Reshma Kodavali... signed Reshma'*. In addition, internal filters only recognise *reshmakodavali@gmail.com*. Connecting any external user causes the agent to impersonate Reshma and fail to detect the new user's own emails. The prompt and sender detection must be made fully dynamic.

### Blocker 4: Lack of Multi-Tenant Database Isolation (SQLite)

The SQLite database (*aura_stats.db*) contains a single *processed_emails* table with no *user_id* or tenant partitioning. If multiple users connect, User B's dashboard displays User A's email subjects, statistics, and replies. Furthermore, SQLite experiences database locking errors when multiple Gunicorn/Uvicorn workers write concurrently.

### Blocker 5: In-Memory Background Threading vs. Distributed Queues

The agent runs as a daemon thread (*threading.Thread*) polling the inbox every 60 seconds. In modern containerized production hosting (AWS ECS, Google Cloud Run, Render), worker instances restart or autoscale, causing the thread to terminate or multiple instances to spawn duplicate polling jobs that double-reply to emails. A distributed Redis + Celery worker queue is required.

### Blocker 6: Absence of Safety Net, Draft Review & Quota Controls

The current prototype immediately sends AI-generated replies directly to external senders. Outbound auto-replies to mailing lists, customer support tickets, or vacation auto-responders risk creating infinite email loops. A mandatory 'Draft First' mode and daily OpenAI token cost caps must be enforced.

# 3. Production Implementation Roadmap (Phases 1 - 4)

Step-by-step technical plan to transform the local prototype into a secure, commercial-grade platform.

## PHASE 1: Multi-Tenancy & Database Modernization (Weeks 1 - 2)

**PostgreSQL Migration:** Replace SQLite with managed PostgreSQL (Supabase, Neon, or AWS RDS) with PgBouncer connection pooling.

**Relational Multi-Tenant Schema:**

- users: uuid, email, full_name, plan_tier, created_at, is_active.
- oauth_credentials: user_id, encrypted_token, encrypted_refresh_token, expiry, scopes.
- email_logs: id, user_id, message_id, thread_id, sender, subject, classification, priority, reply_status, reply_text.
- user_settings: user_id, custom_signature, tone_mode (Formal/Warm/Concise), whitelist_senders, auto_send_enabled.

**Dynamic Prompting:** Replace hardcoded Reshma prompts with parameterized templates: *'You are an executive assistant for {user.full_name}. Write replies in {user.tone} style and sign as {user.signature}.'*

**Tenant Query Isolation:** Enforce WHERE user_id = current_user.id on all statistics, email views, and reply queries.

## PHASE 2: Security Hardening & Zero-Trust Token Vault (Week 3)

**Token Encryption at Rest:** Encrypt OAuth refresh tokens before saving to database using AES-256 GCM (Fernet or AWS KMS). Never expose tokens in cookies.

**Server-Side Redis Sessions:** Replace signed client cookies with server-side Redis sessions (Flask-Session). Set Secure=True, HttpOnly=True, SameSite=Lax.

**Secret Management:** Remove OAUTHLIB_INSECURE_TRANSPORT. Load client secrets and API keys from Google Secret Manager / AWS SSM.

**CSRF & Security Headers:** Implement Flask-WTF CSRFProtect across all routes. Add Content Security Policy (CSP), X-Frame-Options: DENY, and HSTS headers.

**API Rate Limiting:** Deploy Flask-Limiter backed by Redis to prevent automated endpoint flooding (e.g. max 10 manual triage runs/hr).

## PHASE 3: Asynchronous Scalability & Gmail Webhooks (Week 4)

**Celery + Redis Worker Architecture:** Offload heavy LLM triage calls to Celery background workers; keep Flask web requests fast and non-blocking.

**Google Cloud Pub/Sub Webhooks:** Register Gmail Push Notifications (users.watch) to receive instant email notifications instead of polling every 60s.

**Distributed Idempotency (Redlock):** Use Redis distributed locks keyed on (user_id, message_id) to ensure an email is never replied to twice under concurrent execution.

**Resilient LLM Execution:** Implement retry exponential backoff with Tenacity, token budgeting, and fallback models (gpt-4o-mini with Claude 3.5 Haiku fallback).

## PHASE 4: Google OAuth Verification & Legal Compliance (Weeks 5 - 6)

**Custom Domain & SSL:** Provision branded domain (e.g., https://auraemail.com) with Cloudflare / AWS Route53 SSL.

**Hosted Legal Compliance Pages:** Create dedicated /privacy-policy and /terms-of-service disclosing compliance with the *Google API Services User Data Policy* (Limited Use disclosure).

**Search Console Verification:** Complete DNS TXT record ownership verification in Google Search Console.

**CASA Tier 2 Security Assessment:** Complete the third-party CASA security scan and submit proof of compliance to Google Trust & Safety.

**OAuth Consent Demo Video:** Record and submit the required YouTube video showing the complete consent flow and demonstrating scope usage.

## 4. Production Implementation Roadmap (Phases 5 - 6)

### PHASE 5: Commercial SaaS Features, UX & Monetization (Weeks 7 - 8)

**'Draft First' Safe Mode:** Default all accounts to creating Gmail Drafts instead of direct sending, allowing users to review AI replies before sending.

**Tone & Persona Customizer:** Allow users to select their communication tone (Executive Concise, Warm Professional, Casual Founder).

**Real-Time UI Updates:** Replace static dashboard polling with Server-Sent Events (SSE) or WebSockets to show live triage activity feeds.

**Stripe Billing Integration:** Implement subscription plans (Free: 50 emails/mo, Pro: 1,000 emails/mo, Enterprise: Unlimited). Enforce monthly usage quotas.

### PHASE 6: Cloud Deployment, CI/CD & Observability (Week 9)

**Production WSGI Containerization:** Multi-stage Dockerfile packaging Gunicorn WSGI server with Uvicorn workers behind Nginx reverse proxy.

**Serverless Cloud Hosting:** Deploy to Google Cloud Run or AWS ECS Fargate with automated auto-scaling (0 to N instances).

**Automated CI/CD Pipeline:** GitHub Actions workflow executing pytest suites, security linting (Bandit, Flake8), and automatic staging deployments.

**Full-Stack Observability:** Sentry integration for real-time error tracking; Prometheus & Grafana for latency and LLM token cost telemetry.

## 5. Architecture Comparison: Prototype vs. Production

| System Dimension | Current Prototype (web_app.py) | Target Production Architecture |
|---|---|---|
| **Database** | Single SQLite file (*aura_stats.db*), no user isolation | Managed PostgreSQL (Supabase/RDS) + row-level security |
| **OAuth Token Vault** | Plaintext in client-side session cookie | AES-256 encrypted at rest in DB vault + server-side Redis sessions |
| **Background Jobs** | In-memory Python thread (*threading.Thread*) polling 60s | Celery worker pool + Redis queue + Google Cloud Pub/Sub Webhooks |
| **User Personalization** | Hardcoded to Reshma Kodavali in system prompt | Dynamic user profile injection (name, tone, signature, custom rules) |
| **Safety & Oversight** | Direct auto-send only; no human review mechanism | 'Drafts First' mode, whitelist/blacklist, confidence thresholds |
| **Web Server** | Flask built-in development server (Werkzeug) | Gunicorn WSGI server with Gevent/Gthread behind Nginx / Cloud Run |
| **Google Compliance** | Unverified test app (capped at 100 manual test users) | Verified OAuth App + CASA Tier 2 Security Assessment certified |

## 6. Pre-Flight Go-Live Launch Checklist

[ ] **Security:** Rotate all developer API keys and verify zero tokens reside in cookies or git history.

[ ] **Database:** Apply PostgreSQL migration scripts and verify row-level security / tenant constraints.

[ ] **OAuth:** Complete Google Cloud OAuth verification and ensure consent screen shows verified domain.

[ ] **Safety Net:** Default all new user accounts to 'Draft Mode' to prevent unintended outbound email replies.

[ ] **Billing:** Test Stripe webhook events (checkout.session.completed, customer.subscription.deleted).

[ ] **Monitoring:** Verify Sentry error captures and configure alert thresholds for OpenAI rate limits.